

1.

O que é encapsulamento?

Encapsulamento é um princípio da Programação Orientada a Objetos que protege os dados de uma classe, permitindo que eles sejam acessados e modificados apenas por métodos controlados da própria classe. Isso ajuda a manter a segurança e a organização do código.

O que significa encapsular dados em uma classe?

Encapsular dados significa proteger os atributos de uma classe, permitindo que eles sejam acessados e modificados apenas por métodos controlados da própria classe.

Qual é a diferença entre um atributo public, protected e private?

public: pode ser acessado de qualquer lugar do código.

protected: pode ser acessado pela própria classe e pelas classes que herdam dela.

private: só pode ser acessado dentro da própria classe.

Por que não é boa prática deixar todos os atributos como public?

Porque qualquer parte do programa poderia alterar os dados diretamente, o que pode causar erros e deixar o código menos seguro e mais difícil de manter.

2.

Para que servem os métodos get e set?

Os métodos get e set são utilizados para acessar e modificar atributos privados de uma classe de forma controlada, seguindo o princípio do encapsulamento.

Por que usamos getAtributo() ao invés de acessar \$objeto->atributo diretamente?

Porque os atributos geralmente são privados e não podem ser acessados diretamente fora da classe. O getter permite acessar o valor de forma controlada.

O que um método set pode fazer além de simplesmente atribuir um valor?

Ele pode validar os dados antes de armazená-los, impedindo valores inválidos.

Dê um exemplo prático de validação dentro de um set.

```
public function setIdade($idade){  
    if($idade > 0){  
        $this->idade = $idade;  
    }  
}
```

3. Verdadeiro ou Falso

Um atributo private pode ser lido diretamente fora da classe.

Falso. Atributos private só podem ser acessados dentro da própria classe.

Getters e setters só fazem sentido quando os atributos são private.

Verdadeiro. Eles permitem acessar e modificar atributos protegidos.

Encapsulamento aumenta a segurança e a manutenção do código.

Verdadeiro. Ele protege os dados e facilita alterações futuras.

É possível ter um getter sem ter um setter para o mesmo atributo.

Verdadeiro. Assim o valor pode ser consultado, mas não alterado.

4.

Erro: acesso a atributo privado

Erro

```
echo $p->nome;
```

Explicação

O atributo \$nome é private e não pode ser acessado diretamente fora da classe.

Código corrigido

```
<?php

class Produto {

    private $nome;
    private $preco;

    public function __construct($nome, $preco) {
        $this->nome = $nome;
        $this->preco = $preco;
    }

    public function getNome() {
        return $this->nome;
    }
}

$p = new Produto('Teclado', 150.00);

echo $p->getNome();

?>
```

5.

Erro: uso de variável sem \$this

Erro

```
echo 'Aluno: ' . $nome . ' - Nota: ' . $nota;
```

Explicação

Os atributos da classe devem ser acessados usando `$this->`.

Código corrigido

```
<?php

class Aluno {
```

```

private $nome;
private $nota;

public function __construct($nome, $nota) {
    $this->nome = $nome;
    $this->nota = $nota;
}

public function exibir() {
    echo 'Aluno: ' . $this->nome . ' - Nota: ' . $this->nota;
}
}

$a = new Aluno('Carlos', 8.5);
$a->exibir();

?>

```

6.

Erro: setter não bloqueia valor inválido

Erro

O método exibe a mensagem de erro, mas mesmo assim salva a idade.

Código corrigido

```

<?php

class Pessoa {

    private $idade;

    public function setIdade($idade) {

        if ($idade < 0) {
            echo 'Idade invalida!';
            return;
        }

        $this->idade = $idade;
    }
}

```

```
    public function getIdade() {  
        return $this->idade;  
    }  
}
```

```
$p = new Pessoa();  
$p->setIdade(-5);
```

```
?>
```

7.

Erro: instanciação sem new

Erro

```
$c = Carro('Fusca');
```

Explicação

Para criar objetos é obrigatório utilizar a palavra-chave new.

Código corrigido

```
<?php
```

```
class Carro {
```

```
    private $modelo;
```

```
    public function __construct($modelo) {  
        $this->modelo = $modelo;  
    }
```

```
    public function getModelo() {  
        return $this->modelo;  
    }  
}
```

```
$c = new Carro('Fusca');
```

```
echo $c->getModelo();
```

```
?>
```

8.

<?php

```
class Livro {

    private $titulo;
    private $autor;
    private $paginas;

    public function __construct($titulo, $autor, $paginas){
        $this->titulo = $titulo;
        $this->autor = $autor;
        $this->paginas = $paginas;
    }

    public function getTitulo(){
        return $this->titulo;
    }

    public function setTitulo($titulo){
        $this->titulo = $titulo;
    }

    public function getAutor(){
        return $this->autor;
    }

    public function setAutor($autor){
        $this->autor = $autor;
    }

    public function getPaginas(){
        return $this->paginas;
    }

    public function setPaginas($paginas){
        $this->paginas = $paginas;
    }

    public function exhibir(){
        echo "Título: ".$this->titulo."<br>";
        echo "Autor: ".$this->autor."<br>";
        echo "Páginas: ".$this->paginas."<br><br>";
    }
}
```

```
$l1 = new Livro("Dom Casmurro","Machado de Assis",256);  
$l2 = new Livro("O Pequeno Príncipe","Antoine de Saint-Exupéry",96);
```

```
$l1->exibir();  
$l2->exibir();
```

```
?>
```

9.

```
<?php
```

```
class ContaBancaria {  
  
    private $titular;  
    private $saldo;  
  
    public function __construct($titular){  
        $this->titular = $titular;  
        $this->saldo = 0;  
    }  
  
    public function depositar($valor){  
        if($valor > 0){  
            $this->saldo += $valor;  
        }  
    }  
  
    public function sacar($valor){  
        if($valor > $this->saldo){  
            echo "Saldo insuficiente";  
            return;  
        }  
  
        $this->saldo -= $valor;  
    }  
  
    public function getSaldo(){  
        return $this->saldo;  
    }  
  
    public function getTitular(){  
        return $this->titular;  
    }  
}
```

```
$conta = new ContaBancaria("Geovana");

$conta->depositar(500);
$conta->sacar(200);

echo "Saldo final: R$ ".$conta->getSaldo();

?>
```

10.

```
<?php

class Aluno {

    private $nome;
    private $nota1;
    private $nota2;

    public function __construct($nome,$nota1,$nota2){
        $this->nome = $nome;
        $this->nota1 = $nota1;
        $this->nota2 = $nota2;
    }

    public function calcularMedia(){
        return ($this->nota1 + $this->nota2) / 2;
    }

    public function situacao(){
        if($this->calcularMedia() >= 5){
            echo "Aprovado";
        }else{
            echo "Reprovado";
        }
    }
}

$a = new Aluno("Carlos",8,6);

echo "Média: ".$a->calcularMedia()."<br>";
$a->situacao();

?>
```


11.

<?php

```
class Retangulo {

    private $largura;
    private $altura;

    public function __construct($largura,$altura){

        if($largura > 0 && $altura > 0){
            $this->largura = $largura;
            $this->altura = $altura;
        }
    }

    public function calcularArea(){
        return $this->largura * $this->altura;
    }

    public function calcularPerimetro(){
        return 2 * ($this->largura + $this->altura);
    }
}

$r1 = new Retangulo(10,5);
$r2 = new Retangulo(8,4);

if($r1->calcularArea() > $r2->calcularArea()){
    echo "Retângulo 1 possui maior área";
}else{
    echo "Retângulo 2 possui maior área";
}

?>
```

12.

<?php

```
class Funcionario {

    private $nome;
    private $cargo;
```

```

private $salario;

public function __construct($nome,$cargo,$salario){
    $this->nome = $nome;
    $this->cargo = $cargo;
    $this->salario = $salario;
}

public function getNome(){
    return $this->nome;
}

public function getCargo(){
    return $this->cargo;
}

public function getSalario(){
    return $this->salario;
}

public function aumentarSalario($percentual){
    $this->salario = $this->salario * (1 + $percentual / 100);
}

public function exibir(){
    echo "Nome: ".$this->nome."<br>";
    echo "Cargo: ".$this->cargo."<br>";
    echo "Salário: R$ ".number_format($this->salario,2,"","")."<br><br>";
}
}

$f = new Funcionario("Ana","Gerente",3000);

$f->exibir();

$f->aumentarSalario(15);

$f->exibir();

?>

```

13.

```
<?php
```

```
class Temperatura {
```

```

private $celsius;

public function __construct($celsius){
    $this->celsius = $celsius;
}

public function getCelsius(){
    return $this->celsius;
}

public function setCelsius($celsius){
    $this->celsius = $celsius;
}

public function paraFahrenheit(){
    return ($this->celsius * 9/5) + 32;
}

public function paraKelvin(){
    return $this->celsius + 273.15;
}
}

$t1 = new Temperatura(0);
$t2 = new Temperatura(100);
$t3 = new Temperatura(37);

echo "0°C = ".$t1->paraFahrenheit().°F | ".$t1->paraKelvin().°K<br>";
echo "100°C = ".$t2->paraFahrenheit().°F | ".$t2->paraKelvin().°K<br>";
echo "37°C = ".$t3->paraFahrenheit().°F | ".$t3->paraKelvin().°K<br>";

?>

```

14.

```
<?php
```

```

class Estoque {

    private $produtos = [];

    public function adicionarProduto($nome,$quantidade){

        $this->produtos[] = [
            'nome' => $nome,

```

```

        'qtd' => $quantidade
    ];
}

public function listarProdutos(){

    foreach($this->produtos as $p){
        echo $p['nome']. " : ".$p['qtd']. "<br>";
    }
}

public function totalItens(){

    $total = 0;

    foreach($this->produtos as $p){
        $total += $p['qtd'];
    }

    return $total;
}
}

$e = new Estoque();

$e->adicionarProduto("Teclado",5);
$e->adicionarProduto("Mouse", 10);
$e->adicionarProduto("Monitor",2);

$e->listarProdutos();

echo "Total de itens: ".$e->totalItens();

?>

```

15.

```

<?php

class Contato {

    private $nome;
    private $telefone;

    public function __construct($nome,$telefone){
        $this->nome = $nome;
    }
}

```

```

        $this->telefone = $telefone;
    }

    public function getNome(){
        return $this->nome;
    }

    public function getTelefone(){
        return $this->telefone;
    }
}

class Agenda {

    private $contatos = [];

    public function adicionarContato($nome,$telefone){

        $c = new Contato($nome,$telefone);

        $this->contatos[] = $c;
    }

    public function listar(){

        foreach($this->contatos as $c){

            echo $c->getNome(). " - " . $c->getTelefone(). "<br>";
        }
    }
}

$agenda = new Agenda();

$agenda->adicionarContato("Ana","99999-1111");
$agenda->adicionarContato("Carlos","99999-2222");
$agenda->adicionarContato("Maria","99999-3333");

$agenda->listar();

?>

```